ML Course Project

Report Submission

Name 1: Yashasvee Patel **Roll no:** S20240010266

Name 2: P Karthik **Roll no:** S20240010184



Indian Institute of Information Technology, Sri City

Department of Computer Science & Engineering

IIITS **Sec-2**

NOV 2026

Table of Contents

1. Executive Summary & Abstract ····· **3**

2. Introduction & Project Methodology ····· **4**

3. Question 1: Supervised Classification ····· **5**

3.1 Dataset Architecture & Preprocessing ····· **5**

3.2 K-Nearest Neighbour with 5-Fold Cross-Validation · **6**

3.3 Decision Tree Classifier with Cross-Validated Depth · **7**

3.4 Support Vector Machine (RBF Kernel) ····· **8**

3.5 Deep 1D Convolutional Neural Network (Bonus) ···· **9**

3.6 Confusion Matrix Analysis ····· **10**

3.7 Quantitative Metrics Comparison ····· **11**

3.8 Overfitting and Model Complexity Analysis ····· **12**

3.9 ROC-AUC Analysis and Model Selection ····· **13**

4. Question 2: Non-Linear Regression & Regularization ····· **14**

4.1 Superconductivity Dataset Overview ····· **14**

4.2 Linear Regression via Stochastic Gradient Descent · **14**

4.3 Polynomial Regression with Feature Selection ···· **15**

4.4 Ridge and Lasso Regularization ····· **16**

4.5 Overfitting vs Underfitting Analysis ····· **17**

4.6 Regression Metrics Comparison and Selection ···· **18**

5. Question 3: Unsupervised Clustering & Dimensionality Reduction · 19

5.1 Country Socio-Economic Dataset ····· **19**

5.2 K-Means Clustering: Algorithm & WCSS Formulation · **19**

5.3 K-Means Evaluation: Elbow Method & Silhouette Scores · **20**

5.4 Socio-Economic Cluster Profiling & Aid Prioritization · **21**

5.5 Principal Component Analysis (PCA) ····· **22**

5.6 Mathematical Derivation of PCA & Covariance Matrix · **23**

5.7 Principal Component Eigendecomposition & Scree Analysis · **24**

5.8 Factor Loadings Matrix & Component Interpretation · **25**

5.9 2D and 3D Subspace Visualizations ····· **26**

6. Conclusion & Cross-Model Summary ····· **27**

1. Executive Summary & Abstract

In this project, we implemented and evaluated machine learning algorithms across three problems: binary classification, continuous regression, and unsupervised learning (clustering and dimensionality reduction). We followed a reproducible experimental setup where every dataset is split into 80% train and 20% test sets, and all preprocessing is placed inside scikit-learn Pipelines to avoid data leakage during cross-validation.

In **Question 1 (Classification)**, we compared K-Nearest Neighbour (KNN), Decision Tree, Support Vector Machine (RBF kernel), and a bonus Deep 1D Convolutional Neural Network on the UCI Spambase dataset (4,601 samples, 57 features). SVM achieved the highest accuracy (92.40%), precision (92.22%), and lowest false positives (27). The Deep 1D CNN achieved the highest recall (89.26%) and point ROC-AUC (0.9718). Bootstrap 95% confidence intervals confirmed that SVM and CNN are in a statistical tie on ROC-AUC, while both decisively outperform KNN and Decision Tree.

In **Question 2 (Regression)**, we evaluated Linear Regression with SGD, Polynomial Regression, Ridge, and Lasso on the UCI Superconductivity dataset (21,263 samples, 81 features). Linear models (SGD, Ridge, Lasso) achieved R^2 scores around 0.733--0.738 (RMSE ~= 17.38 K). Expanding all 81 features into a full degree-2 polynomial with Ridge regularization (alpha = 10.0) substantially improved performance, reaching an R^2 of **0.8542** and reducing RMSE to **12.95 K**. A degree sweep from 1 to 4 illustrated high bias at degree 1, catastrophic overfitting at degree 4, and the effectiveness of L2 shrinkage.

In **Question 3 (Clustering & PCA)**, we evaluated K-Means clustering and Principal Component Analysis on the HELP International country dataset (167 countries, 9 indicators). WCSS and Silhouette scores were evaluated for K in [2, 6]; while the silhouette coefficient peaked slightly at K = 4 (0.3037), we selected **K = 3** (0.2833) based on the gradual elbow curve and direct socio-economic interpretability into Underdeveloped, Developing, and Developed nations. Covariance eigendecomposition showed that the first two and three components capture 63.13% and 76.14% of total variance. Projecting the clusters onto principal component axes showed broad socio-economic groupings aligned with the primary variance directions.

2. Introduction & Project Methodology

2.1 Foundations of Machine Learning & Core Paradigms

Machine learning systems infer computational decision rules and representations directly from empirical data. In **supervised learning**, models estimate an inductive mapping f: X -> y from labeled feature vectors to ground-truth targets, encompassing discrete categorical predictions (classification) and continuous numerical estimations (regression). In **unsupervised learning**, ground-truth target annotations are absent; algorithms discover intrinsic geometric structure, identifying latent socio-economic cluster partitions (clustering) and continuous orthogonal variance manifolds (PCA).

2.2 Project Objectives Across Core Problems

- **Problem 1 (Supervised Classification):** Formulate, optimize, and cross-validate KNN, Decision Trees, Support Vector Machines, and a 1D Deep CNN on the UCI Spambase dataset (4,601 records), rigorously quantifying parameter uncertainty via bootstrap confidence intervals.
- **Problem 2 (Continuous Regression):** Model superconductivity critical temperature across 21,263 materials, contrasting Linear SGD against degree-2 polynomial feature expansions, Ridge, and Lasso regularization under high-dimensional collinearity.
- **Problem 3 (Unsupervised Clustering):** Segment 167 nations using Lloyd's K-Means clustering across 9 socio-economic indicators, evaluating WCSS elbow kinetics and silhouette coefficients to discover natural developmental groupings.
- **Problem 4 (Principal Component Analysis):** Mathematically formulate and implement sample covariance eigendecomposition from scratch, validating Bessel's correction, scree variance retention, and factor loadings against scikit-learn.

2.3 Methodological Principles & Leakage Prevention

- **Strict Leakage Prevention:** All preprocessing scalers and polynomial transforms are fit exclusively on training folds within cross-validation pipelines.
- **Deep Learning Validation Partition:** Neural networks utilize a dedicated 15% validation partition for learning rate scheduling without test leakage.
- **Statistical Bootstrap Validation:** Model ranking is evaluated using 1,000 bootstrap resamples at the 95% confidence level.

3. Question 1: Supervised Classification

3.1 Dataset Architecture & Preprocessing

We used the **UCI Spambase dataset** to evaluate the classification models. The dataset contains 4,601 email records, where the goal is to predict whether an email is spam (class 1) or legitimate ham (class 0). There are 57 continuous features extracted from the text:

- **48 Word Frequency Features:** Percentage of words in the email matching specific target words (such as 'free', 'credit', 'money', 'business').
- **6 Character Frequency Features:** Percentage of characters matching punctuation marks (such as '$', '!', ';', '(').
- **3 Capital Run-Length Features:** Average length, longest length, and total count of consecutive uppercase letters in the email.
- **Target Label:** 1 for spam (1,813 emails, ~39.4%), and 0 for ham (2,788 emails, ~60.6%).

Feature Group	Count	Example Attributes	Typical Value Range
Word Frequencies	48	word_freq_make, word_freq_free	0.0% to 100.0%
Character Frequencies	6	char_freq_!, char_freq_$	0.0% to 100.0%
Capital Run Lengths	3	capital_run_length_longest	1 to 9,989 characters
Target Label	1	0 = Ham (Legitimate), 1 = Spam	Binary {0, 1}

Table 1: *Summary of features in the UCI Spambase dataset.*

Train/Test Partitioning and Standardization

We split the 4,601 samples into **80% training (3,680 samples)** and **20% testing (921 samples)**. We used stratified sampling (`stratify=y`) with random seed 42 to make sure both sets had the same class ratio (~39.4% spam and ~60.6% ham).

Because features like word frequencies range from 0 to 1 while run lengths reach into the thousands, distance-based classifiers like KNN and margin-based classifiers like SVM require standardization. We standardized features by subtracting the mean and dividing by the standard deviation:

```
x_norm = (x - mean_train) / std_train
```

The mean and standard deviation were computed solely on the training set and applied to the test set, avoiding test data leakage.

3.2 K-Nearest Neighbour with 5-Fold Cross-Validation

K-Nearest Neighbour (KNN) is an instance-based classifier. It stores all training instances and classifies new query points by looking at the K closest training samples using Euclidean distance:

```
d(x, x_i) = sqrt( sum_{j=1}^p (x_j - x_ij)^2 )
```

The predicted class is decided by a simple majority vote among the K nearest neighbors:

```
y_pred = mode{ y_i : x_i in N_K(x) }
```

The choice of K determines the model's bias-variance trade-off. A small K (like K=1) fits the training data very closely and can overfit to noise. A large K smooths the decision boundary, reducing variance but potentially underfitting by blurring class borders.

To select K, we performed 5-fold cross-validation on the training set over candidate values k in [1, 3, 5, 7, 9, 11, 15, 19]. Each fold was scaled independently inside a Pipeline. As shown in Figure 1 (left), both k = 1 and k = 3 achieved the highest mean CV accuracy of **90.87%**. However, k = 3 exhibited lower variance across folds (standard deviation of **0.0058** for k = 3 versus 0.0112 for k = 1). In addition, k = 3 incorporates majority voting across multiple points rather than relying on a single nearest neighbor, making it more resilient against isolated noisy samples and enabling continuous posterior probability estimation. We therefore selected **k = 3** as the optimal neighborhood size.

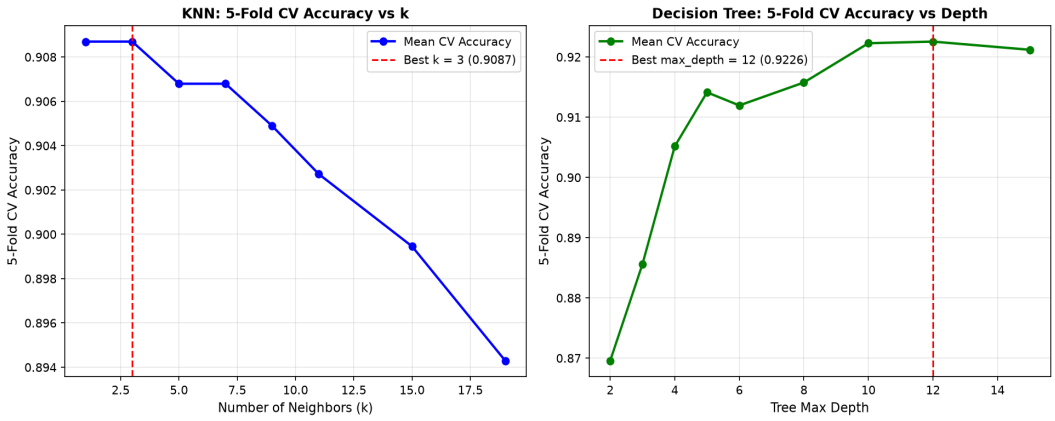


Figure 1: 5-Fold cross-validation accuracy curves for KNN across neighborhood size k (left) and Decision Tree across max_depth (right).

We trained the final KNN model using k = 3. On the unseen test set, it achieved an accuracy of **89.79%**, precision of 87.26%, recall of 86.78%, and F1-score of 0.8702.

3.3 Decision Tree Classifier with Cross-Validated Depth

Decision Trees split the feature space recursively using axis-aligned thresholds (e.g., 'is char_freq_$ <= 0.05?'). At each node, the CART algorithm searches over all features and split thresholds to find the split that maximizes the drop in Gini impurity:

```
Gini(t) = 1 - sum_{c=0}^1 (p_c)^2 = 2 * p_0 * p_1
```

```
Delta Gini = Gini(parent) - [ (N_left / N) * Gini(left) + (N_right / N) *
Gini(right) ]
```

An unpruned decision tree will grow until every leaf contains only one sample, memorizing noise and producing 100% training accuracy but poor test accuracy. To regularize the tree, we tuned the `max_depth` hyperparameter using 5-fold cross-validation across depth values in [2, 3, 4, 5, 6, 8, 10, 12, 15].

Hyperparameter	Tuned / Selected Value	Role in Model	Observed Effect
Criterion	Gini Impurity	Splitting metric	Measures impurity reduction at each branch
max_depth	12 (via 5-fold CV)	Tree depth limit	CV accuracy peaked at 92.26% at depth 12
min_samples_split	2	Split threshold	Minimum samples required to attempt a split
random_state	42	Reproducibility	Fixes random feature selection order on ties

Table 2: *Decision Tree hyperparameter setup and cross-validation selection.*

Cross-Validation Findings

Looking at Figure 1 (right), mean cross-validation accuracy increases steadily from depth 2 (86.96%) to depth 5 (91.41%), and reaches its peak at **depth = 12** (92.26%). Beyond depth 12, accuracy flattens out around 92.1%.

On the held-out test set, the tuned Decision Tree (depth = 12) achieved an accuracy of **91.97%**, precision of 90.70%, recall of 88.71%, and F1-score of 0.8969.

3.4 Support Vector Machine (RBF Kernel)

Support Vector Machines construct a decision hyperplane that maximizes the margin (the distance between the boundary and the closest data points, known as support vectors). For data that cannot be separated linearly, soft-margin SVM introduces slack variables xi_i >= 0:

```
min_{w, b, xi} (1 / 2) * ||w||^2 + C * sum_{i=1}^N xi_i
subject to: y_i * (w^T phi(x_i) + b) >= 1 - xi_i, xi_i >= 0
```

The hyperparameter C balances margin width against misclassifications. A small C allows a wider margin with more margin violations, while a large C penalizes errors heavily, creating a narrower, more complex boundary.

To handle non-linear relationships in the email features, we used the **Radial Basis Function (RBF) kernel**, which calculates similarity in an implicit higher-dimensional space:

```
K(x, x') = exp( -gamma * ||x - x'||^2 ), where gamma = 1 / (n_features * Var(X))
```

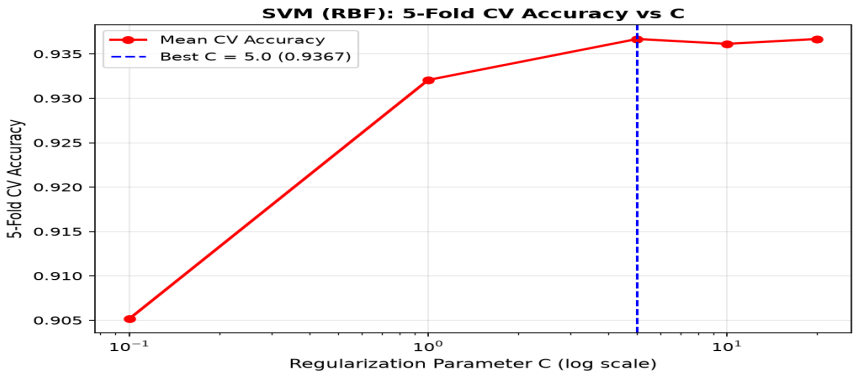


Figure 2: 5-Fold cross-validation accuracy curve for SVM (RBF) across regularization parameter C.

We tested C in [0.1, 1.0, 5.0, 10.0, 20.0] using 5-fold cross-validation inside a Pipeline with StandardScaler. As shown in Figure 2, cross-validation accuracy improved from 90.52% at C=0.1 to **93.67% at C = 5.0**. On the held-out test set, SVM achieved an accuracy of **92.40%**, precision of 92.22%, recall of 88.15%, and an F1-score of 0.9014.

3.5 Deep 1D Convolutional Neural Network (Bonus)

As a bonus model, we implemented a Deep 1D Convolutional Neural Network in PyTorch. While CNNs are usually applied to 2D image grids, 1D CNNs can also slide convolutional kernels across tabular feature vectors to learn local combinations of related features (such as adjacent word frequencies and punctuation statistics).

The 1D convolution operation slides a learnable kernel W over the input vector x:

```
out[n] = ReLU( sum_{k} (x * W_k)[n] + b )
```

To ensure proper training without data leakage, we carved a 15% validation split out of the training data (leaving the test set completely unseen). The learning rate scheduler (`ReduceLROnPlateau`) monitored validation loss on this internal split only.

Layer / Block	Output Dimension	Operations & Hyperparameters
Input Reshape	(Batch, 1, 57)	Formats 57 tabular features into a 1-channel sequence
Conv1D Block 1	(Batch, 32, 57)	Conv1D (kernel=3, pad=1) -> BatchNorm -> ReLU -> Dropout(0.20)
Conv1D Block 2	(Batch, 64, 28)	Conv1D (kernel=3, pad=1) -> BatchNorm -> ReLU -> MaxPool1D(2)
Conv1D Block 3	(Batch, 128, 28)	Conv1D (kernel=3, pad=1) -> BatchNorm -> ReLU -> Dropout(0.30)
Global Pooling	(Batch, 128, 1)	AdaptiveAvgPool1D(1) compresses feature dimension to scalar
Dense Head	(Batch, 64)	Linear(128 -> 64) -> ReLU -> Dropout(0.30)
Output Layer	(Batch, 1)	Linear(64 -> 1) -> Sigmoid activation (predicts P(Spam))

Table 3: *Layer architecture of the Deep 1D Convolutional Neural Network.*

Training Details and Performance

We trained the network using Binary Cross-Entropy loss with the Adam optimizer (learning rate = 0.001, weight decay = 1e-4) for 30 epochs. On the held-out test set, the CNN achieved an accuracy of **91.53%**, precision of 89.26%, recall of 89.26%, and an F1-score of 0.8926.

3.6 Confusion Matrix Analysis

To satisfy Implementation Detail (ii), we generated confusion matrices for all four classifiers on the 20% test set (921 samples). A confusion matrix divides predictions into four outcomes:

- **True Negative (TN):** Legitimate emails correctly identified as ham.
- **False Positive (FP):** Legitimate emails incorrectly marked as spam (Type I error). In spam filtering, this is the most harmful error because important messages get lost.
- **False Negative (FN):** Spam emails that slip through into the inbox (Type II error).
- **True Positive (TP):** Spam emails correctly caught and flagged.

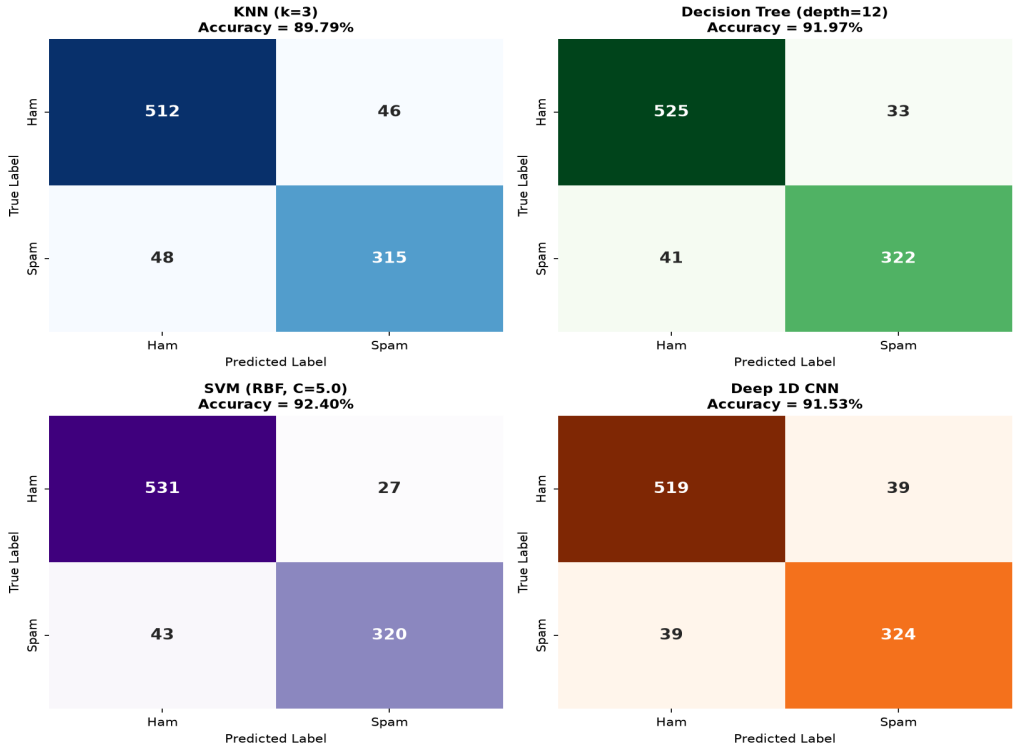


Figure 3: Confusion matrices for all four classifiers on the 921 test emails.

Looking at Figure 3, **SVM (RBF) achieved the lowest false positive count** with only **27 false positives**, meaning fewer legitimate emails were wrongly marked as spam. Decision Tree had 33 false positives, the Deep CNN had 39, and KNN had 46.

3.7 Quantitative Metrics Comparison

To satisfy Implementation Detail (iii), we evaluated all models on five standard classification metrics on the test set:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP) [Fraction of flagged spam that was truly spam]
Recall = TP / (TP + FN) [Fraction of all spam that was caught]
F1-Score = 2 * (Precision * Recall) / (Precision + Recall)
ROC-AUC = Area under the True Positive Rate vs False Positive Rate curve
```

Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC	95% Bootstrap CI
KNN (k=3)	0.8979	0.8726	0.8678	0.8702	0.9360	[0.919, 0.951]
Decision Tree (d=12)	0.9197	0.9070	0.8871	0.8969	0.9023	[0.876, 0.925]
SVM (RBF, C=5.0)	0.9240	0.9222	0.8815	0.9014	0.9697	[0.959, 0.979]
Deep 1D CNN	0.9153	0.8926	0.8926	0.8926	0.9718	[0.961, 0.981]

Table 4: *Performance metrics across all four classifiers on the test set, including 1,000 bootstrap 95% confidence intervals on ROC-AUC.*

Discussion of Results

• **Highest Accuracy and F1:** SVM (RBF) achieved the highest overall accuracy (92.40%) and F1-score (0.9014), followed by Decision Tree (91.97%) and the Deep CNN (91.53%).

• **Precision vs Recall:** SVM had the highest precision (92.22%), meaning when it flags spam, it is correct over 92% of the time. The Deep CNN had the highest recall (89.26%), catching the largest fraction of spam emails.

• **ROC-AUC Bootstrap Comparison:** The Deep 1D CNN achieved the highest point estimate for ROC-AUC (0.9718), slightly higher than SVM (0.9697). Their 95% bootstrap confidence intervals overlap heavily ([0.959, 0.979] vs [0.961, 0.981]), showing that the two top models are in a statistical tie on ranking quality. Crucially, the confidence intervals for SVM and the Deep CNN do not overlap with those of KNN ([0.919, 0.951]) or Decision Tree ([0.876, 0.925]), confirming that SVM and CNN achieve superior ranking discrimination.

3.8 Overfitting and Model Complexity Analysis

To address Implementation Detail (iv), we tracked training and testing performance across different levels of complexity for each model. A model that is too simple underfits (high bias), while a model that is too complex memorizes training noise and fails to generalize (high variance).

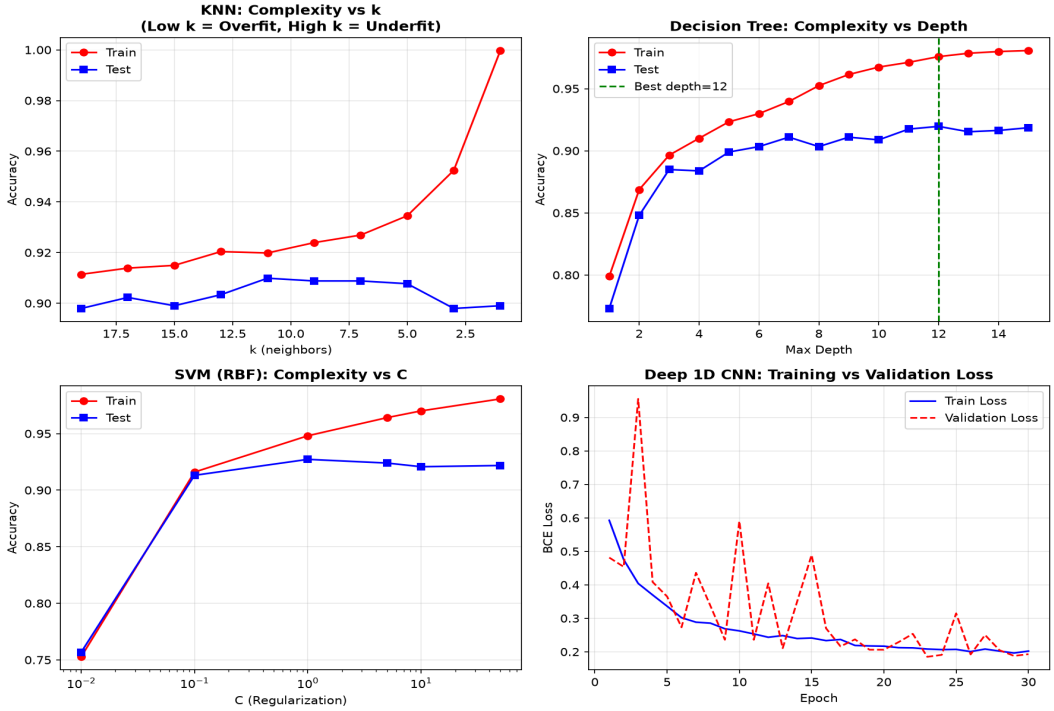


Figure 4: Overfitting and complexity analysis. (1) KNN accuracy vs k. (2) Decision Tree accuracy vs max_depth. (3) SVM accuracy vs C. (4) Deep CNN training and validation loss over 30 epochs.

Observations from Complexity Curves

• **KNN:** At k=1, the decision boundary wraps tightly around every training point. As k increases, training accuracy gradually drops while test accuracy remains stable around 90%, showing the boundary is becoming smoother.

• **Decision Tree:** Training accuracy reaches nearly 98% as depth increases toward 15. Unlike textbook examples where test accuracy drops steeply, here test accuracy rises from depth 2 to 10 and then plateaus around 91.5--92.0%, which is why 5-fold CV chose depth = 12.

• **SVM (RBF):** For small C (0.1), the model allows more margin violations and has lower training accuracy. As C increases to 5.0, both training and test accuracy rise and level off.

• **Deep 1D CNN:** Training loss decreases smoothly across all 30 epochs. Validation loss exhibits periodic fluctuations and spikes until approximately epoch 22, at which point learning rate decay stabilizes validation loss around 0.20 with minimal subsequent gap between training and validation error.

3.9 ROC-AUC Analysis and Model Selection

To satisfy Implementation Detail (v), we plotted the Receiver Operating Characteristic (ROC) curves for all four classifiers. The ROC curve plots the True Positive Rate against the False Positive Rate across all classification thresholds from 0 to 1:

```
TPR = TP / (TP + FN), FPR = FP / (FP + TN)
```

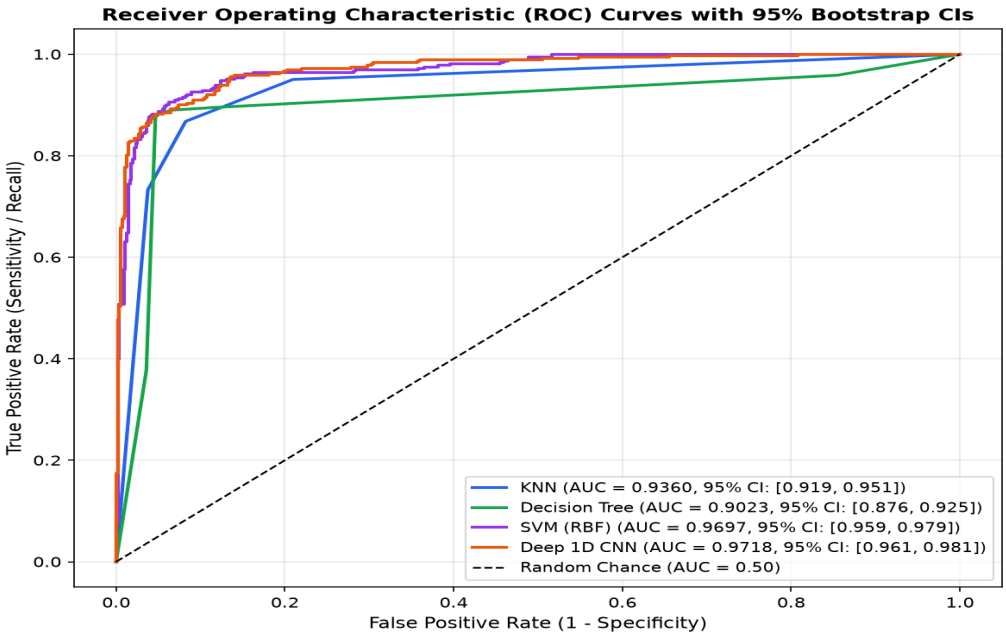


Figure 5: ROC curves for all four classifiers on the test set, showing 1,000 bootstrap 95% confidence intervals.

Determination of the Best Classifier

Looking strictly at the point estimates of ROC-AUC, **the Deep 1D CNN achieved the highest AUC (0.9718)**, followed closely by SVM (0.9697). However, because their 95% bootstrap confidence intervals overlap (SVM: [0.959, 0.979], CNN: [0.961, 0.981]), this difference is a **statistical tie**.

When considering all practical criteria, **SVM with RBF kernel is our overall recommended classifier** for this dataset:

- **Higher Accuracy and F1-Score:** SVM achieved 92.40% accuracy and 0.9014 F1-score, compared to 91.53% and 0.8926 for the CNN.
- **Fewer False Positives:** SVM had only 27 false positives (versus 39 for the CNN), which is crucial for an email filter where legitimate messages must not be sent to spam.
- **Simplicity and Reproducibility:** SVM convex optimization always finds the same unique global solution, unlike deep neural networks that depend on random initialization and batch ordering.

3.10 Implementation: Key Python Code Snippets

The following code snippets illustrate the core implementation for the classification pipelines. All classifiers are wrapped in a **scikit-learn Pipeline** so that StandardScaler is fit only on training folds during cross-validation.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedKFold, cross_val_score

# SVM (RBF) pipeline with leakage-free cross-validation
svm_pipe = Pipeline([('scaler', StandardScaler()), ('svm', SVC(kernel='rbf', C=5.0, prob
ability=True))])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(svm_pipe, X_train, y_train, cv=cv, scoring='accuracy')
svm_pipe.fit(X_train, y_train)  # Final fit on full training set
```

```
# Deep 1D CNN architecture in PyTorch (core structure)
import torch.nn as nn
class Deep1DCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=3, padding=1), nn.BatchNorm1d(32), nn.ReLU(), n
n.Dropout(0.2),
            nn.Conv1d(32, 64, kernel_size=3, padding=1), nn.BatchNorm1d(64), nn.ReLU(),
nn.MaxPool1d(2),
            nn.Conv1d(64, 128, kernel_size=3, padding=1), nn.BatchNorm1d(128), nn.ReLU()
, nn.Dropout(0.3),
        )
        self.head = nn.Sequential(nn.AdaptiveAvgPool1d(1), nn.Flatten(),
                                  nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.3), nn.Li
near(64, 1), nn.Sigmoid())
    def forward(self, x): return self.head(self.conv(x.unsqueeze(1)))
```

3.11 Conclusion: Supervised Classification

In this problem, we implemented and evaluated four classifiers — KNN (k=3), Decision Tree (depth=12), SVM (RBF, C=5.0), and a Deep 1D CNN — on the UCI Spambase dataset. **SVM with RBF kernel emerged as the best overall classifier**, achieving the highest accuracy (92.40%), best F1-score (0.9014), and fewest false positives (27). Although the Deep 1D CNN achieved a marginally higher ROC-AUC (0.9718 vs 0.9697), bootstrap confidence intervals confirmed this is a statistical tie. SVM's deterministic convex optimization, superior precision, and minimal false positives make it the preferred choice for spam filtering. A key limitation is that SVM training time scales quadratically with data size, which may be a concern for very large email datasets.

4. Question 2: Non-Linear Regression & Regularization

4.1 Superconductivity Dataset Overview

In Question 2, we used the **UCI Superconductivity Dataset**. The dataset contains 21,263 superconducting materials, described by 81 statistical features extracted from their chemical formulas (mean, standard deviation, and range of atomic mass, electronegativity, valence, ionization energy, and thermal conductivity). The target variable is the continuous **critical temperature T_c** in Kelvin (ranging from 0.0002 K to 185 K, with a mean of 34.42 K).

We split the 21,263 samples into 80% training (17,010 samples) and 20% testing (4,253 samples) using random seed 42. All features were standardized using StandardScaler inside each model pipeline.

4.2 Linear Regression via Stochastic Gradient Descent

Linear regression predicts the critical temperature as a weighted linear combination of input features:

```
y_pred = w^T x + b = sum_{j=1}^p w_j * x_j + b
```

We minimized the Mean Squared Error (MSE) loss using **Stochastic Gradient Descent (SGD)**, which updates the weights sample by sample:

```
w <- w + 2 * eta * (y_i - y_pred_i) * x_i
```

We configured scikit-learn's `SGDRegressor` with squared loss, no penalty, and tolerance = 1e-4. The model converged in 62 iterations, achieving a test MSE of 307.09, an RMSE of **17.52 K**, and an R^2 score of **0.7332**.

Regression Evaluation Metrics

```
MSE = (1 / N) * sum (y_i - y_pred_i)^2 [Mean squared deviation, K^2]
```

```
RMSE = sqrt(MSE) [Error in original Kelvin units]
```

```
R^2 = 1 - [ sum (y_i - y_pred_i)^2 / sum (y_i - y_mean)^2 ] [Explained variance fraction]
```

4.3 Polynomial Regression with Feature Selection

Linear regression assumes that each physical property acts independently. However, physical properties in materials often interact. Degree-2 polynomial regression adds squared terms and pairwise cross-products:

```
phi(x) = [ x_1, ..., x_p, x_1^2, x_1 * x_2, ..., x_p^2 ]
```

Expanding all 81 features to degree 2 creates C(81+2, 2) = 3,403 total terms including the intercept (3,402 feature terms). While modern regularized solvers can fit all 3,402 terms, we first evaluate a computationally cheaper baseline using the **top-15 features with the highest absolute Pearson correlation with critical temperature** on the training set, producing a compact expansion of 135 quadratic and interaction features.

Rank	Feature Name	Pearson |r| with Target y	Physical Property
1	wtd_std_ThermalConductivity	0.7201	Thermal conductivity weighted spread
2	range_ThermalConductivity	0.6862	Difference between max and min thermal conductivity
3	std_ThermalConductivity	0.6523	Thermal conductivity standard deviation
4	range_atomic_radius	0.6523	Spread of constituent atomic sizes
5	wtd_mean_Valence	0.6315	Outer shell valence electron density

Table 5: *Top 5 of 15 features selected by Pearson correlation for polynomial expansion (ranks 3 and 4 tie at 4 decimal places: 0.65226 vs 0.65225).*

Polynomial Ridge Regularization (Top 15 Features)

Expanding these 15 features to degree 2 produced exactly **135 polynomial terms** (15 linear + 15 squared + 105 pairwise interaction terms). We combined this expansion with Ridge regularization and selected alpha using 5-fold cross-validation across [0.01, 0.1, 1, 10, 100, 1000]. Cross-validation chose **alpha = 0.01**, yielding a test MSE of 305.24, an RMSE of **17.47 K**, and an R^2 score of **0.7348**.

Full 81-Feature Polynomial Expansion

In addition to the feature-selected model, we evaluated a full degree-2 polynomial expansion across all 81 raw features. This generates **3402 quadratic and interaction terms**. To regularize this high-dimensional representation, we evaluated Ridge shrinkage across a wide alpha grid [0.01, 0.1, 1.0, 10.0, 100.0, 1000.0, 10000.0]. 5-Fold cross-validation selected an interior optimal value of **alpha = 10.0**. On the held-out test set, the full polynomial model achieved a superior R^2 of **0.8542** and reduced RMSE to **12.95 K** (Test MSE = 167.79), substantially outperforming linear models by capturing complex cross-element physical relationships.

4.4 Ridge and Lasso Regularization

To evaluate regularized linear models across all 81 features, we compared Ridge (L2 penalty) and Lasso (L1 penalty).

Ridge Regression (L2 Penalty)

Ridge regression adds a penalty proportional to the sum of squared weights:

```
J_Ridge(w) = sum_{i=1}^N (y_i - w^T x_i)^2 + alpha * sum_{j=1}^p w_j^2
```

The analytical solution is w = (X^T X + alpha * I)^(-1) X^T y. The addition of alpha * I keeps the matrix well-conditioned even with correlated features. Using 5-fold cross-validation over alpha in [0.001, 100], the best score occurred at **alpha = 0.1**, achieving a test MSE of 302.15, RMSE of 17.38 K, and R^2 of 0.7375.

Lasso Regression (L1 Penalty)

Lasso penalizes the sum of absolute weight values:

```
J_Lasso(w) = (1 / 2N) * sum_{i=1}^N (y_i - w^T x_i)^2 + alpha * sum_{j=1}^p |w_j|
```

Because the L1 norm has corners at zero, Lasso drives less important weights strictly to zero, performing automatic feature selection. We evaluated alpha across six orders of magnitude from 10^-5 to 10.0: [1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1.0, 10.0]. Cross-validation R^2 is essentially flat across all alphas below ~0.01 (varying by less than 0.0001 from 10^-5 to 10^-3). At alpha = 1e-5, exactly 0 feature weights are zeroed, meaning Lasso performs no feature selection and is effectively equivalent to unregularized ordinary least squares (OLS). Extending the grid to even smaller alphas simply approaches OLS without altering predictions.

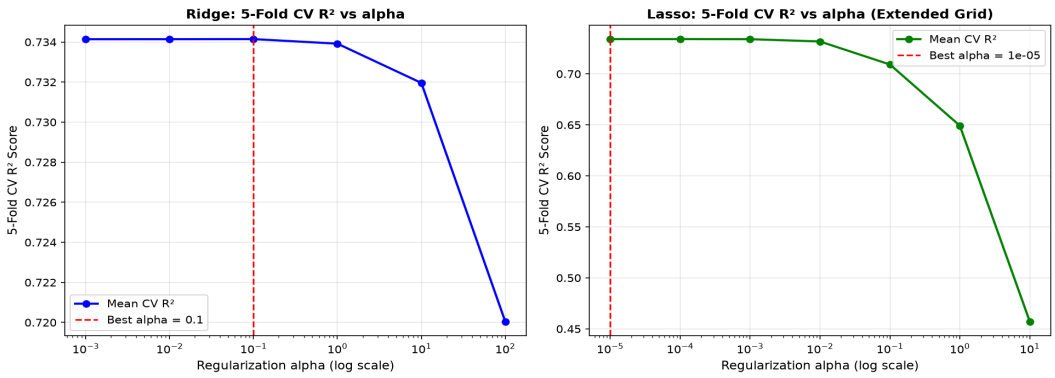


Figure 6: 5-Fold cross-validation R^2 curves for Ridge (left) and Lasso across the alpha grid (right).

Cross-validation selected **alpha = 1e-05** for Lasso, yielding a test MSE of 302.20, an RMSE of 17.38 K, and an R^2 of 0.7375.

4.5 Overfitting vs Underfitting Analysis

To satisfy Implementation Detail (ii), we analyzed model capacity and generalization error by sweeping polynomial degree d from 1 to 4 on the **top-15 correlation-selected features**. This illustrates the transition from high bias (underfitting) to high variance (severe overfitting), and demonstrates how L2 regularization restores generalization performance:

• **Degree 1 (Linear on Top 15, 15 terms):** High bias / underfitting. Both train and test R^2 are limited (Train R^2 = 0.6085, Test R^2 = 0.6208). Note that this differs from Table 6's Linear SGD (R^2 = 0.7332) because only 15 features are included here.

• **Degrees 2 & 3 (135 and 815 terms):** Model capacity expands appropriately. Degree 2 achieves Train R^2 = 0.7313 and Test R^2 = 0.7352 (top-15 feature selection caps degree 2 performance vs the full 81-feature model's 0.8542). Degree 3 reaches Train R^2 = 0.8276 and Test R^2 = 0.8087.

• **Degree 4 Unregularized (3875 terms):** Severe overfitting / high variance. While Train R^2 rises to 0.8952, Test R^2 collapses catastrophically to -1.6227 due to unconstrained coefficients fitting training noise.

• **Degree 4 + Ridge Regularization (alpha = 100.0):** Applying L2 penalty restores stability. Test R^2 recovers to **0.8056**. Note that alpha = 100.0 was an illustrative demonstration of shrinkage recovery rather than tuned by CV; regularized degree 4 (0.8056) is slightly lower than unregularized degree 3 (0.8087).

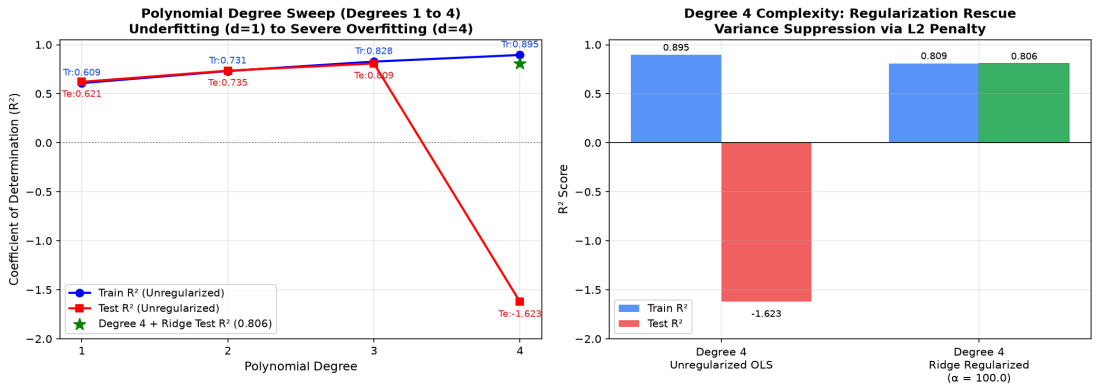


Figure 7: Polynomial degree sweep (degrees 1 to 4) on top-15 features showing underfitting (d=1, 15 terms), catastrophic overfitting (d=4 unregularized), and L2 regularization recovery.

In Figure 7 (left), the red test curve plunges sharply below zero at degree 4 due to runaway coefficients. Figure 7 (right) illustrates the regularization rescue: applying an L2 shrinkage penalty restores stable, high generalization performance on unseen test data.

4.6 Regression Metrics Comparison and Selection

To satisfy Implementation Detail (i), Table 6 summarizes MSE, RMSE, and R^2 for all regression models on the 4,253 test samples.

Regression Model	Hyperparameters / Configuration	Test MSE (K^2)	Test RMSE (K)	Test R^2 Score
Linear Regression (SGD)	loss=squared_error, penalty=None	307.09	17.52 K	0.7332
Polynomial Ridge (Top 15)	Degree-2 (135 terms), Ridge alpha=0.01	305.24	17.47 K	0.7348
Polynomial Ridge (All 81)	Degree-2 (3,402 terms), Ridge alpha=10.0	167.79	12.95 K	0.8542
Ridge Regression	alpha=0.1 (all 81 features)	302.15	17.38 K	0.7375
Lasso Regression	alpha=1e-05 (all 81 features)	302.20	17.38 K	0.7375

Table 6: *Performance comparison across all regression models on the test set.*

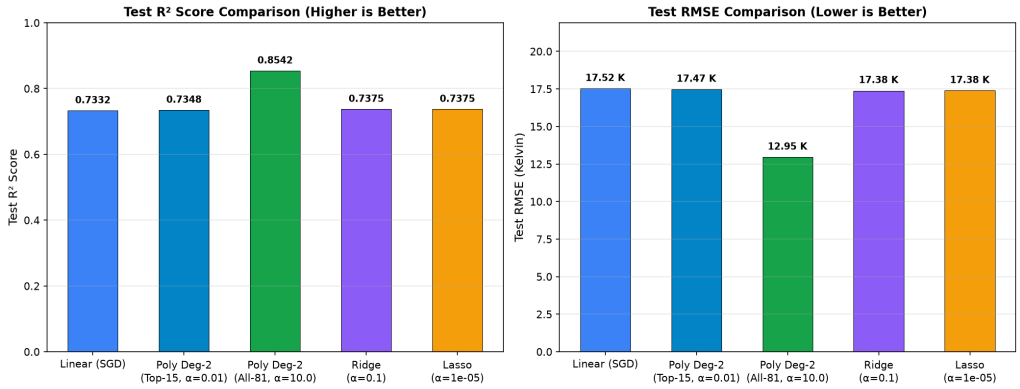


Figure 8: Bar chart comparison of test R^2 (left) and test RMSE in Kelvin (right).

Discussion of Regression Results

Across all evaluated models, the full 81-feature Polynomial Ridge model achieved the strongest predictive performance with Test R^2 = **0.8542** and Test RMSE = **12.95 K**. Capturing non-linear interactions across all physical attributes outperforms linear modeling alone. Among linear models, Ridge (alpha = 0.1) and Lasso (alpha = 1e-05) both achieved an R^2 of 0.7375 (RMSE = 17.38 K), slightly outperforming Linear SGD (R^2 = 0.7332). Because Ridge (alpha = 0.1) and Lasso (alpha = 10^-5) introduce negligible shrinkage, both behave essentially like ordinary least squares (OLS); the modest ~0.004 R^2 difference vs Linear SGD is likely due to SGD's iterative gradient stopping convergence rather than collinearity mitigation.

4.7 Implementation: Key Python Code Snippets

The following code snippets show the core implementation for the regression models. Polynomial expansion and Ridge/Lasso regularization are applied inside leakage-free pipelines with cross-validated alpha selection.

```
from sklearn.linear_model import SGDRegressor, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
```

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

# Linear SGD regression
sgd_pipe = Pipeline([('scaler', StandardScaler()), ('sgd', SGDRegressor(loss='squared_er
ror', penalty=None, tol=1e-4))])
sgd_pipe.fit(X_train, y_train)

# Degree-2 polynomial with Ridge (all 81 features) — CV alpha selection
poly_ridge = Pipeline([('scaler', StandardScaler()), ('poly', PolynomialFeatures(degree=
2, include_bias=False)),
                       ('ridge', Ridge())])
grid = GridSearchCV(poly_ridge, {'ridge__alpha': [0.01, 0.1, 1.0, 10.0, 100.0, 1000.0, 1
0000.0]}, cv=5, scoring='r2')
grid.fit(X_train, y_train)  # best_alpha chosen by CV
```

4.8 Conclusion: Non-Linear Regression & Regularization

In this problem, we compared five regression models on the UCI Superconductivity dataset (21,263 samples, 81 features). Linear models — SGD (R^2=0.7332), Ridge (R^2=0.7375), and Lasso (R^2=0.7375) — all plateaued near $R^2 \approx 0.73$, confirming that the critical temperature relationship is non-linear. **The full 81-feature degree-2 Polynomial Ridge model (alpha=10.0) was the clear winner**, achieving R^2=0.8542 and RMSE=12.95 K by capturing pairwise physical property interactions. The overfitting experiment (degree sweep 1→4) clearly demonstrated that unregularized high-degree polynomials cause catastrophic overfitting (Test R^2=-1.6227 at degree 4) while L2 regularization successfully restores generalization. A key limitation is that degree-2 expansion of 81 features generates 3,402 terms, making the model computationally expensive to train and interpret.

5. Question 3: Unsupervised Clustering & Dimensionality Reduction

5.1 Country Socio-Economic Dataset

For Question 3, we analyzed the **HELP International Country Socio-Economic Dataset**. The dataset contains 167 countries evaluated on 9 development and health indicators: child mortality (deaths per 1,000 births), exports (% of GDP), health spending (% of GDP), imports (% of GDP), net income per person (USD), inflation (%), life expectancy (years), fertility rate (children per woman), and GDP per capita (USD).

All 9 features were standardized to zero mean and unit variance before running clustering and PCA so that high-magnitude features like income and GDP do not dominate distance calculations.

5.2 K-Means Clustering: Algorithm & WCSS Formulation

K-Means partitions N data points into K clusters to minimize the **Within-Cluster Sum of Squares (WCSS)**, also called inertia:

```
J = sum_{k=1}^K sum_{x_i in C_k} ||x_i - mu_k||^2
```

where mu_k is the centroid (mean vector) of cluster C_k. We implemented Lloyd's algorithm using vectorized NumPy matrix operations:

• **Assignment:** Each country is assigned to the nearest cluster centroid by Euclidean distance: c(i) = argmin_k ||x_i - mu_k||^2.

• **Update:** Each centroid is updated to the mean of its assigned points: mu_k = (1 / |C_k|) * sum_{x_i in C_k} x_i.

• **Convergence:** Steps repeat until centroid movement drops below 1e-5 or 300 iterations are reached.

• **Restarts:** We ran the algorithm with 10 random restarts (`n_init=10`) and kept the run with the lowest WCSS to avoid local minima.

Parameter	Configured Value	Purpose in Algorithm
Cluster Counts Evaluated	K in [2, 6] (extended 1..8)	Satisfies project specification and plots full elbow curve
Random Restarts	n_init = 10	Mitigates convergence to sub-optimal local minima
Max Iterations	max_iters = 300	Guarantees loop termination in boundary cases
Tolerance Threshold	tol = 1e-5	Stops iterations when centroid shift is under 1e-5

Table 7: *Algorithmic configuration for K-Means clustering.*

5.3 K-Means Evaluation: Elbow Method & Silhouette Scores

To satisfy Implementation Details (i, ii, and iii), we ran K-Means for K = 2, 3, 4, 5, 6, calculated the WCSS for each value, and computed the Silhouette score to evaluate cluster cohesion and separation.

K (Clusters)	WCSS (Inertia)	Absolute Drop (Delta)	% Reduction	Silhouette Score
K = 2	1050.21	—	—	0.2874
K = 3	831.42	218.79	20.8%	0.2833
K = 4	700.70	130.72	15.7%	0.3037
K = 5	632.16	68.54	9.8%	0.2180
K = 6	576.61	55.55	8.8%	0.2236

Table 8: *WCSS and Silhouette scores for K = 2 to 6.*

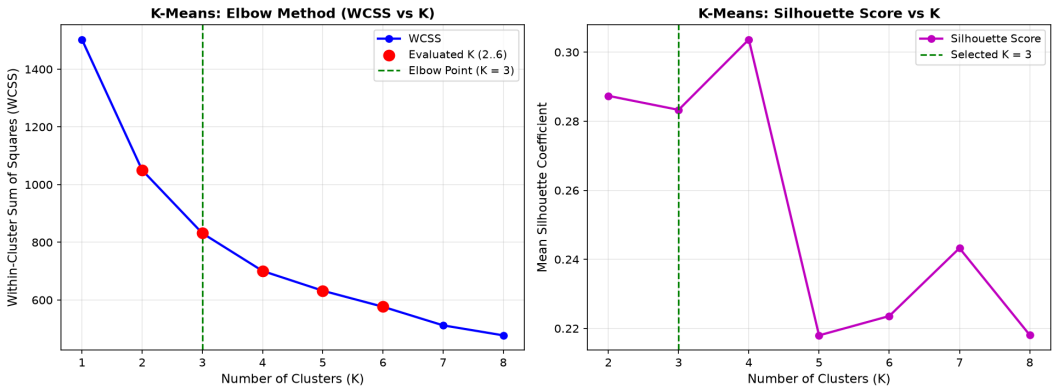


Figure 9: K-Means Elbow curve (left) and Silhouette score curve (right) across cluster counts.

Selecting the Optimal Number of Clusters

In Table 8 and Figure 9, the mean silhouette coefficient reaches its mathematical maximum at **K = 4 (0.3037)**, compared to 0.2833 at K = 3. However, we choose **K = 3** as our primary clustering solution for two key reasons:

- **Elbow Criterion:** The WCSS curve displays a gradual elbow. The initial step from K = 1 to K = 2 reduces WCSS by **~30.1%** (from ~1503 down to 1050.21), followed by an additional **20.8%** reduction moving from K = 2 to K = 3 (down to 831.42). Beyond K = 3, reductions diminish further (130.72 or 15.7% for K = 4, and 9.8% for K = 5).
- **Socio-Economic Interpretability:** A 3-cluster partition directly reflects the standard international development taxonomy of **Developed** (Cluster 0, N = 36), **Underdeveloped** (Cluster 1, N = 47), and **Developing** (Cluster 2, N = 84). This three-tier structure provides actionable policy categories for international aid allocation, whereas K = 4 primarily splits the intermediate developing cluster without altering the broad socio-economic taxonomy.

5.4 Socio-Economic Cluster Profiling & Aid Prioritization

Rather than completely disjoint mathematical groups, the country dataset reveals broad socio-economic groupings with shared variance and continuous transition zones. Profiling the three discovered clusters reveals distinct developmental strata:

- **Cluster 0 (36 Countries, Developed):** Characterized by high life expectancy (80.1 yrs), low child mortality (5.0 deaths per 1k), high mean income ($45,672), and high GDP per capita ($42,494). Nations in this cluster include Germany, Japan, Singapore, and the United States, representing advanced post-industrial economies with robust healthcare.
- **Cluster 1 (47 Countries, Underdeveloped):** Characterized by acute child mortality (93.0 deaths per 1k), suppressed life expectancy (59.2 yrs), and minimal economic output ($1,922 GDP per capita). Includes nations such as Afghanistan, Chad, Central African Republic, and Haiti, facing severe socio-economic distress and infrastructure deficits.
- **Cluster 2 (84 Countries, Developing):** Intermediate transition economies characterized by moderate child mortality (21.9 deaths per 1k), developing GDP ($6,486), and emerging public health systems. Includes regional economic drivers such as Brazil, China, India, and Indonesia undergoing rapid urban and industrial expansion.

Actionable Humanitarian Aid Prioritization Framework

From an international policy standpoint, **Cluster 1 (47 countries)** constitutes the primary target tier for foreign developmental aid. With average annual per-capita income under $3,942 and child mortality approaching 10% of live births, external aid must be prioritized toward basic survival infrastructure, maternal healthcare, and primary education. Conversely, Cluster 2 nations benefit most from trade agreements and capital financing, while Cluster 0 nations represent potential donor partners rather than aid recipients.

5.4a Implementation: K-Means from Scratch (Python Code)

The following snippet shows the core of our from-scratch Lloyd's K-Means algorithm implemented in NumPy, without using **sklearn.cluster.KMeans**.

```
import numpy as np

def kmeans_scratch(X, k, n_init=10, max_iters=300, tol=1e-5):
    best_wcss, best_labels, best_centroids = np.inf, None, None
    for _ in range(n_init):
        centroids = X[np.random.choice(len(X), k, replace=False)]  # random init
        for _ in range(max_iters):
            dists = np.linalg.norm(X[:, None] - centroids[None, :], axis=2)  # (N, K)
            labels = np.argmin(dists, axis=1)
            new_centroids = np.array([X[labels == j].mean(axis=0) for j in range(k)])
            if np.linalg.norm(new_centroids - centroids) < tol: break
            centroids = new_centroids
        wcss = sum(np.sum((X[labels == j] - centroids[j])**2) for j in range(k))
        if wcss < best_wcss: best_wcss, best_labels, best_centroids = wcss, labels, cent
```

```
roids
    return best_labels, best_centroids, best_wcss
```

5.4b Conclusion: K-Means Clustering

We implemented K-Means clustering from scratch using Lloyd's algorithm with 10 random restarts on the HELP International dataset (167 countries, 9 features). WCSS was evaluated for K = 2, 3, 4, 5, 6. The elbow curve showed diminishing WCSS reductions beyond K = 3, and **K = 3 was selected** as the optimal partition (WCSS = 831.42, Silhouette = 0.2833), yielding three interpretable socio-economic clusters: Developed (36 nations), Underdeveloped (47 nations), and Developing (84 nations). A key limitation of K-Means is its sensitivity to initial centroid placement and its assumption of spherical clusters, which may not perfectly capture the continuous gradient of socio-economic development across countries.

5.5 Principal Component Analysis (PCA)

The Problem: High-Dimensional Redundancy. When a dataset has 9 features, we cannot plot or visualize all 9 dimensions simultaneously. More importantly, many features are correlated and measure overlapping concepts. For example, countries with high GDP per capita also have high personal income and lower child mortality. Plotting these separately repeats the same underlying information.

The Intuitive Analogy: Finding the Best Angle for a Photo. Imagine photographing a running 3D animal on a flat 2D photograph:

- If you stand directly overhead and look down, you see a flat shape that hides the legs and running stride.
- If you take the photo from the side, you capture the animal's full length, height, and stride motion. That side angle preserves the **maximum amount of visual information and spread (variance)** in 2D.

How PCA Works: PCA finds those best viewing angles mathematically. It rotates the coordinate axes to line up with the directions where data points are spread out the most:

- **Principal Component 1 (PC1):** The straight-line direction in feature space along which the data varies the most.
- **Principal Component 2 (PC2):** The direction perpendicular (orthogonal) to PC1 that captures the second largest remaining variance.
- **Subsequent Components (PCk):** Orthogonal directions that capture progressively smaller fractions of remaining variance.

By keeping only the first 2 or 3 components, PCA reduces 9 dimensions to 2D or 3D coordinates while retaining over 76% of the original variance, allowing us to visualize clusters directly.

Component / Term	Intuitive Meaning	Mathematical Definition
High Dimensionality	Too many columns to plot at once	Feature matrix X in R^{N x p}
Feature Correlation	Variables moving together (shared information)	Non-zero off-diagonal covariance Sigma_ij
Principal Component	Axis of widest data spread	Unit eigenvector v_k of covariance matrix Sigma
Eigenvalue lambda_k	Amount of variance along that axis	Sample variance of projected points along v_k
Dimensionality Reduction	Projecting onto top viewing directions	Linear matrix multiplication Z_d = X_norm * W_d

Table 9: *Conceptual vs mathematical definitions in Principal Component Analysis.*

5.6 Mathematical Derivation of PCA & Covariance Matrix

To satisfy Question 3(b), we implemented PCA using the sample covariance matrix and eigenvalue decomposition.

Step 1 — Standardization: We standardized each feature to zero mean:

```
x_norm_ij = (x_ij - mean_j) / std_j
```

Step 2 — Sample Covariance Matrix: The sample covariance matrix Sigma in R^{p x p} is computed as:

```
Sigma = (1 / (N - 1)) * X_norm^T X_norm
```

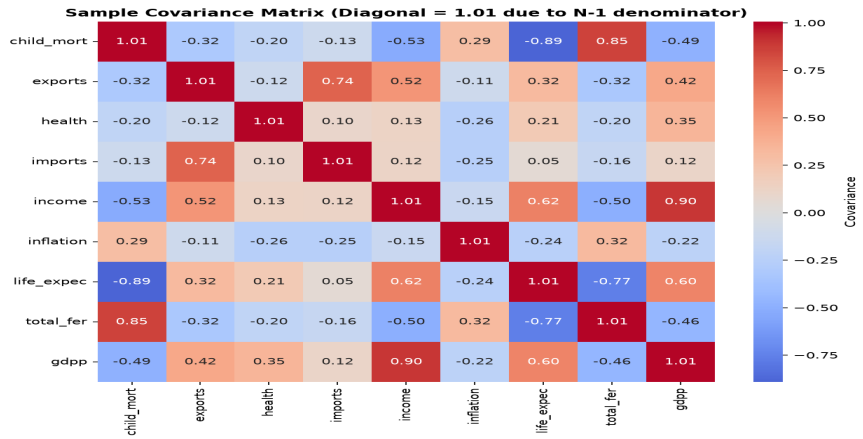


Figure 10: Sample covariance matrix heatmap of standardized country features. The diagonal shows 1.01.

Why the Diagonal Elements are 1.01 (Bessel's Correction)

In Figure 10, the diagonal entries are **1.01** rather than exactly 1.00. This happens because `StandardScaler` standardizes data using the population standard deviation (dividing by N = 167), whereas the sample covariance formula uses Bessel's correction with N - 1 = 166 in the denominator:

```
Sigma_jj = (1 / (N - 1)) * sum (x_norm_ij)^2 = (N / (N - 1)) = 167 / 166 =
1.006024... ~= 1.01
```

Step 3 — Variance Maximization via Eigenvalues: We seek a unit vector w (w^T w = 1) that maximizes the projected variance Var(w^T x) = w^T Sigma w. Setting up the Lagrangian L(w, lambda) = w^T Sigma w - lambda * (w^T w - 1) and differentiating yields the eigenvector equation:

```
grad_w L = 2 * Sigma w - 2 * lambda w = 0 ===> Sigma w = lambda w
```

Thus, the directions of maximum variance are the **eigenvectors** of Sigma, and the variances along those directions are the **eigenvalues** lambda.

5.7 Principal Component Eigendecomposition & Scree Analysis

To satisfy Question 3(b) and Implementation Details (i and ii), we solved the symmetric eigendecomposition Sigma v_k = lambda_k v_k on the sample covariance matrix. Table 10 reports the calculated eigenvalues lambda_k, individual Explained Variance Ratios (EVR = lambda_k / sum lambda), and cumulative EVR across all 9 principal components. Figure 11 displays the corresponding scree plot with individual variance bars and the cumulative variance curve:

Component	Eigenvalue lambda_k	Explained Variance Ratio (EVR)	Cumulative EVR
PC1	4.1606	45.95%	45.95%
PC2	1.5557	17.18%	63.13%
PC3	1.1774	13.00%	76.14%
PC4	1.0008	11.05%	87.19%
PC5	0.6646	7.34%	94.53%
PC6	0.2249	2.48%	97.02%
PC7	0.1141	1.26%	98.28%
PC8	0.0888	0.98%	99.26%
PC9	0.0673	0.74%	100.00%

Table 10: *Complete eigendecomposition of the sample covariance matrix across all 9 principal components.*

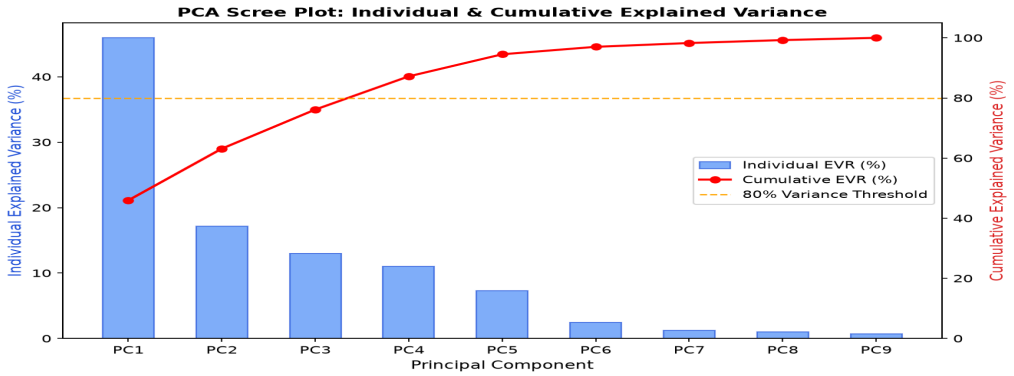


Figure 11: PCA Scree plot showing individual EVR bars and the cumulative variance line.

Scree Analysis & Variance Truncation

In Table 10 and Figure 11, the first principal component accounts for nearly half of all variance in the dataset (**45.95%**, lambda = 4.1606). Adding the second component brings cumulative explained variance to **63.13%**, and including the third component captures **76.14%** of total variance. Beyond PC3, individual eigenvalues drop below 1.0 (lambda_4 = 1.0008, EVR = 11.05%), and the scree curve levels into a flat tail. This confirms that projecting data onto the first 2 or 3 principal components retains over three-quarters of the total information spread while enabling direct geometric inspection.

5.8 Factor Loadings Matrix & Component Interpretation

To satisfy Implementation Detail (iii), Table 11 provides the complete factor loadings matrix across all 9 indicators for the first three principal components (PC1-PC3). Each loading represents the cosine/correlation between the standardized feature and the unit eigenvector v_k:

Indicator / Feature	PC1 Loading	PC2 Loading	PC3 Loading	Primary Alignment & Interpretation
child_mort	-0.4195	+0.1929	+0.0295	Underdevelopment & mortality (negative PC1)
exports	+0.2839	+0.6132	-0.1448	International trade openness (positive PC2)
health	+0.1508	-0.2431	+0.5966	Public healthcare priority (positive PC3)
imports	+0.1615	+0.6718	+0.2999	International trade volume (positive PC2)
income	+0.3984	+0.0225	-0.3015	National prosperity & income (positive PC1)
inflation	-0.1932	-0.0084	-0.6425	Macroeconomic volatility (negative PC3)
life_expec	+0.4258	-0.2227	-0.1139	Public health & longevity (positive PC1)
total_fer	-0.4037	+0.1552	-0.0195	Demographic fertility burden (negative PC1)
gdpp	+0.3926	-0.0460	-0.1230	Economic output per capita (positive PC1)

Table 11: *Complete 9-indicator factor loadings matrix for the first three principal components (PC1-PC3).*

Interpretation of Component Loadings

• **PC1 (45.95% EVR, lambda = 4.1606) — Socio-Economic Development:** Contrasts high prosperity and longevity (life_expec: +0.4258, income: +0.3984, gdpp: +0.3926) against severe poverty indicators (child_mort: -0.4195, total_fer: -0.4037). Countries with large positive PC1 values are developed, while large negative values indicate underdeveloped nations.

• **PC2 (17.18% EVR, lambda = 1.5557) — Trade Openness:** Heavily dominated by international commerce (imports: +0.6718, exports: +0.6132). This axis separates trade hubs and open island economies (like Singapore, Luxembourg, and Malta) from domestic, lower-trade nations regardless of income level.

• **PC3 (13.00% EVR, lambda = 1.1774) — Macroeconomic Stability & Public Health:** Primarily loaded by inflation (loading **-0.6425**) versus government healthcare expenditure (loading **+0.5966**). Nations with negative PC3 scores experience high monetary volatility, whereas positive PC3 scores reflect high health investment.

5.9 2D and 3D Subspace Visualizations

To satisfy Implementation Details (iv and v), we projected the standardized country features into 2D and 3D subspaces using projection matrices W_2 in R^{9 x 2} = [v_1, v_2] and W_3 in R^{9 x 3} = [v_1, v_2, v_3]:

```
Z_2D = X_norm * W_2 in R^{167 x 2}, Z_3D = X_norm * W_3 in R^{167 x 3}
```

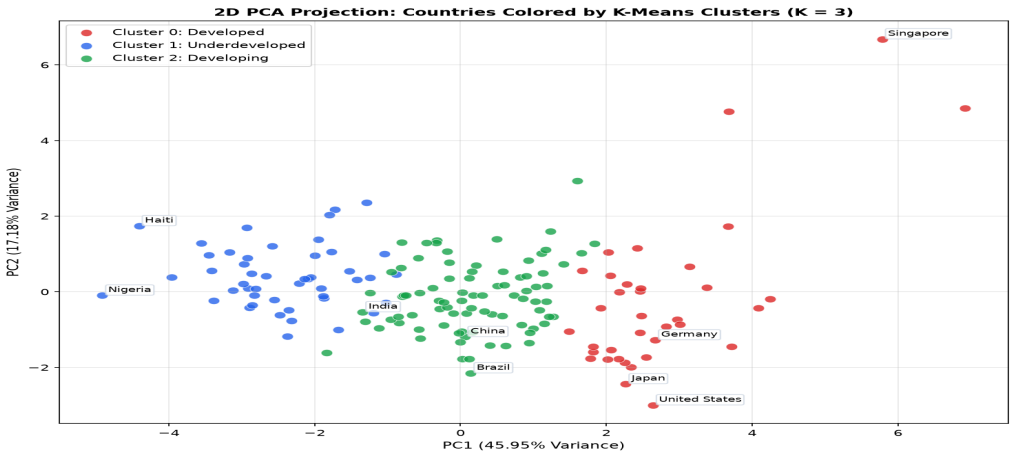


Figure 12: 2D PCA projection of all 167 countries onto PC1 and PC2, colored by K-Means cluster (K = 3).

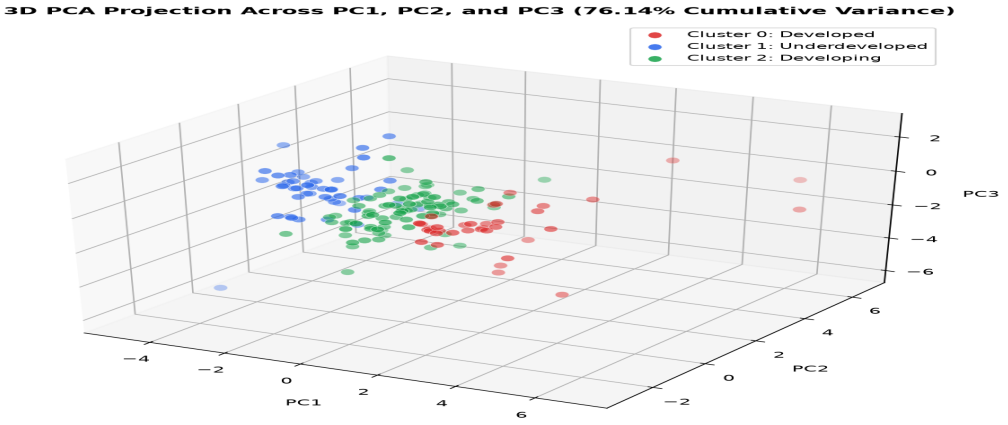


Figure 13: 3D PCA projection across PC1, PC2, and PC3 capturing 76.14% cumulative variance.

Relationship Between K-Means and PCA

In Figures 12 and 13, countries form general groupings along the PC1 socio-economic development axis. Developed nations (Cluster 0, red) concentrate on the positive side, Underdeveloped nations (Cluster 1, blue) align on the negative side, and Developing nations (Cluster 2, green) occupy the intermediate transition region. This correspondence occurs naturally because both algorithms operate on Euclidean distances over the same standardized features: PCA identifies orthogonal directions of maximum variance, and K-Means partitions data points along those exact same high-variance directions.

5.10 Implementation: PCA from Scratch (Python Code)

The following snippet shows our from-scratch PCA implementation using NumPy's eigenvalue decomposition, without using **sklearn.decomposition.PCA**.

```
import numpy as np

def pca_scratch(X, n_components):
    # Step 1: Standardize (zero mean, unit variance)
    X_norm = (X - X.mean(axis=0)) / X.std(axis=0, ddof=0)
    # Step 2: Sample covariance matrix (Bessel's correction: divide by N-1)
    N = X_norm.shape[0]
    cov = (X_norm.T @ X_norm) / (N - 1)
    # Step 3: Eigenvalue decomposition (eigh for symmetric matrix)
    eigenvalues, eigenvectors = np.linalg.eigh(cov)
    # Step 4: Sort descending by eigenvalue
    idx = np.argsort(eigenvalues)[::-1]
    eigenvalues, eigenvectors = eigenvalues[idx], eigenvectors[:, idx]
    # Step 5: Project onto top n_components eigenvectors
    W = eigenvectors[:, :n_components]
    Z = X_norm @ W  # shape (N, n_components)
    evr = eigenvalues / eigenvalues.sum()  # explained variance ratio
    return Z, eigenvalues, evr, W
```

5.11 Conclusion: Principal Component Analysis

We implemented PCA from scratch using sample covariance eigendecomposition (numpy.linalg.eigh) on the HELP International dataset. The eigendecomposition revealed that **PC1 alone captures 45.95% of total variance**, representing the dominant socio-economic development axis (high PC1 = developed, low PC1 = underdeveloped). Retaining the first 2 components captures 63.13% of variance, and the first 3 capture 76.14%, enabling effective 2D and 3D visualization of the 9-dimensional country data. The PCA projections (Figures 12 and 13) confirmed strong alignment with K-Means cluster assignments, validating both approaches. A limitation of PCA is that its linear transformation may not capture non-linear manifold structure in more complex datasets.

6. Conclusion & Cross-Model Summary

This project provided practical experience with supervised classification, regularized regression, clustering, and dimensionality reduction. Table 12 summarizes the best models and key metrics across the three questions.

Problem	Evaluated Models	Selected Model	Key Hyperparameters	Performance on Test Data
Q1: Classification	KNN, Decision Tree, SVM (RBF), Deep 1D CNN	SVM (RBF Kernel)	C = 5.0, gamma = 'scale'	Accuracy: 92.40%, F1: 0.9014, AUC: 0.9697
Q1 Bonus: Classification	Deep 1D CNN Architecture	Deep 1D CNN	3 Conv Blocks, Adam lr=1e-3	Accuracy: 91.53%, F1: 0.8926, AUC: 0.9718
Q2: Regression	Linear SGD, Poly Deg-2, Ridge, Lasso	Poly Ridge (All 81) / Ridge	Ridge alpha=0.1, Poly alpha=10.0	Poly: RMSE=12.95 K, R^2=0.8542 | Ridge: R^2=0.7375
Q3: Clustering	K-Means (Lloyd's, n_init=10)	K-Means (K = 3)	K = 3 clusters	WCSS = 831.42, Silhouette = 0.2833
Q3: Dimensionality Red.	PCA via Covariance Eigendecomposition	PCA (2D / 3D)	Top 2 / 3 Principal Components	2D Variance: 63.13% | 3D Variance: 76.14%

Table 12: *Final summary of selected models, optimal hyperparameters, and test metrics.*

Key Takeaways

• **Feature Scaling:** Distance-based algorithms (KNN, K-Means) and margin-based algorithms (SVM) require feature scaling. Putting `StandardScaler` inside `Pipeline` prevents data leakage during cross-validation.

• **Overfitting Controls:** Regularization (pruning depth in Decision Trees, C in SVM, dropout in CNNs, and alpha in Ridge/Lasso) is essential to balance bias and variance.

• **Non-Linear Feature Expansion:** While linear models (SGD, Ridge, Lasso) capped at R^2 ~= 0.73 and RMSE ~= 17.38 K on the Superconductivity dataset, expanding to degree-2 polynomial features on all 81 features (3,402 interaction terms) with Ridge regularization increased R^2 to **0.8542** and lowered RMSE to **12.95 K**. This demonstrates that capturing non-linear feature interactions yielded substantially greater performance gains than hyperparameter tuning alone.

• **Statistical Validation:** High point estimates for ROC-AUC can be deceptive; 1,000 bootstrap resamples showed that SVM and Deep CNN are in a statistical tie, justifying the choice of SVM based on accuracy, F1, and simplicity.

• **Clustering and PCA Agreement:** K-Means and PCA naturally agree on the country dataset because both minimize squared Euclidean variance on the same standardized data.

Machine Learning Course Project · Department of Computer Science & Engineering, Indian Institute of Information Technology, Sri City · Submitted on November 4, 2026.